个人简历介绍 | 面试 · 述职 · 路演

haswhere

B2B 大宗贸易供应链平台 · 技术经理

Java + Vue 全栈 | 异构架构治理

交易系统 | 供应链金融 | 平台化建设 | 13 年一线技术经验

186-****-3592 | 815841365@qq.com | 郑州 | 2026

我做什么，我的差异化是什么

30 秒内建立第一印象——三个标签要刻在面试官脑子里

我做什么

负责大宗商品 B2B 交易平台的业务中台，管理交易撮合、履约保证、数据服务、规则配置和生态对接五大核心模块；带 20 人团队（3C / 6Java / 5Vue / 5 测试），对接 9+ 外部系统。

我的差异化 —— 三个标签

01

驾驭 C+Java 异构架构

管住黑盒的经典防腐层范式，市场稀缺

02

技术 × 业务深度交叉

懂撮合引擎，也懂大宗贸易全链路

03

三年零资金差错

大额资金场景的金融级可靠性

加分句：我至今保持在一线——方案能不能落地、坑在哪、工作量多少，我能直接判断，不给团队挖坑。

用数据建立可信度

面试官判断段位，先看数字再听故事

平台规模

13 年	一线技术经验（2 年清算 + 11 年本司）
100+	大客户差异化支撑，代码零膨胀
20 人	研发团队 · 3C/6Java/5Vue/5 测试

技术指标

12+	Java 微服务（业务中台）
9+	外部系统对接（银行 / 物流 / 税务…）
4 层	接入 / 网关 / 中台 / 数据 异构架构

业务价值

5 天→ 1.5 天	交易周期
2 个月→ 1 周	新客户上线周期
20→50 笔/月	月交易量翻倍

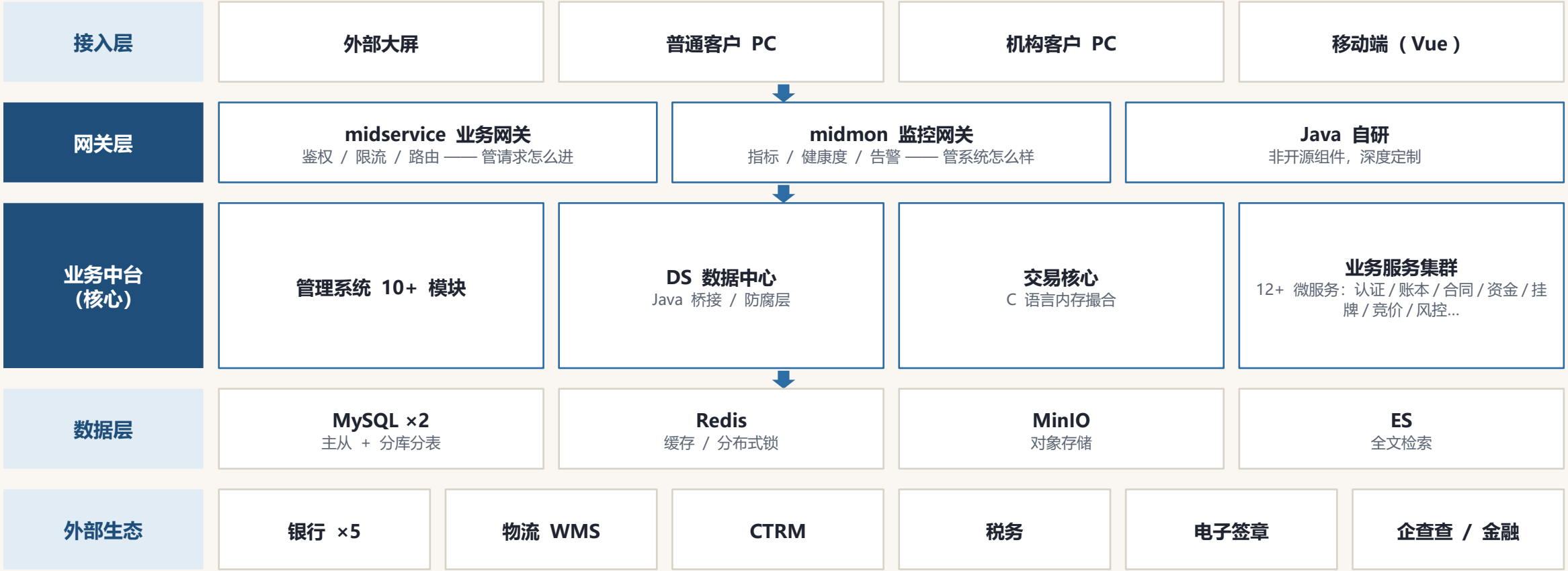
质量指标

3 年	零资金差错
99.95%	外部接口平均可用率
连续多季	冬储高峰零 P0 事故

最有力的一句话：三年零资金差错——这是大额资金场景给我的名片。

四层架构 + 双核驱动 + 9+ 外部生态

先讲分层（30 秒），再展开双核（1 分钟），最后扫外部接口矩阵



讲法要点：每层按「为什么存在 / 不这么做会怎样 / 取舍了什么」讲——两个网关分工一句话说清。

DS 数据中心 + 交易核心：管住黑盒的经典范式

不把 Java 写进 C 撮合引擎，也不把 C 逻辑外泄到 Java 业务层

Java 业务编排层
订单 / 风控 / 结算 / 报表 / 生态对接

↓ Hessian over HTTP （同步） · Socket （异步）

DS 数据中心（Java）
桥接层 / 防腐层 Anti-Corruption Layer —— 协议转换、连接管理、可靠性收敛于一层

↓ TCP 自定义二进制协议

交易核心（C 语言）
单线程内存撮合 · 纳秒级延迟确定性 · 无 GC 停顿 · 异步落库

核心设计理念

- **C 负责「快」**——撮合不能有 GC 停顿
- **Java 负责「稳」**——业务编排灵活可迭代
- **DS 负责「翻译」**——异构复杂度收敛在一层
- **两个团队独立演进**——C 团队与 Java 团队互不阻塞

选型决策：否决「C 撮合引擎重写成 Java」——撮合是资金链路心脏，GC 停顿不可接受；保留 C + DS 防腐层，确定性与可维护性兼得。

驾驭的技术广度

快速扫一遍即可——重点是广度，细节等面试官追问

接入层	Vue 前端（PC + 移动端 + 大屏三端）	Nginx
网关层	midservice 业务网关（Java 自研）	midmon 监控网关
业务中台	Java 微服务（Spring Cloud 生态）	DS 数据中心（Hessian + Socket）	交易核心（C 语言内存撮合）
数据存储	MySQL （主从 / 分库分表）	Redis （缓存 / 分布式锁）	MinIO 对象存储	Elasticsearch
中间件	Kafka （outbox 本地消息表 + 消费幂等）	Canal CDC	XXL-Job	Redisson	Drools 规则引擎
运维部署	Docker + K8s	GitLab CI/CD	Prometheus + Grafana
异构通信	TCP 自定义二进制协议	Hessian 序列化	Socket 长连接
外部对接	银行 / 物流 WMS / CTRM / 税务 / 电子签章 / 企查查 / 供应链金融（9+ 接口）

我管什么、怎么管：守心脏，不管全部

高级技术经理的标志是知道什么该管、什么该放

深度区 —— 直接掌控

- 交易模式（挂牌 / 竞价 / 询价 / 招标 / 基差点价）
- 履约保证（仓单 / 质检 / 交收）
- 数据服务（行情 / 报表 / 对账）
- 规则配置（计费 / 风控 / 促销）
- 生态对接（银行 / 仓储 / 物流 / 税务）

→ **平台耦合最复杂的「心脏」：我直接 review 代码、做架构决策**

治理区 —— 契约管理

◇ **交易核心（C 撮合）**

→ 通过 DS 桥接层 + 可观测性管控

◇ **财务清结算**

→ 通过接口契约 + SLA 管控

◇ **物流执行 / 风控引擎**

→ 通过依赖风险地图管控

→ **管「连线」不管「黑盒内部」：守住接口、 SLA 、爆炸半径**

管理理念：用机制替代亲力亲为。

一笔挂牌交易经过的完整技术链路

把所有技术点串成一条链——中间件不是孤立的

买方下单
行情比价 → 选择挂牌 → 提交订单

↓

网关层 midservice
鉴权 / 限流 / 路由

↓

交易服务（Java）
创建订单
— Redis SETNX 防重复提交 | Redisson 锁防超卖

↓

DS 数据中心 → C 撮合
Hessian → 二进制协议 → 内存撮合
— 撮合结果异步落库

↓

撮合成功 → Kafka
outbox 本地消息表 + 消费幂等
— 异步通知 4 个下游：仓储备货 / 财务记账 / 风控调额度 / 客户通知

↓

Saga 分布式事务
锁库存 → 扣额度 → 创建合同
— 失败反向补偿，保证最终一致

- **规则引擎 Drools** —— 实时计算手续费，热更新不重启
- **Canal CDC** —— binlog 同步缓存服务加速行情网页 | 全链路 traceId + Prometheus + Grafana 可观测

> **金句：这套组合解决三件最难的事——高并发撮合、长周期一致、规则热更。**

一笔大宗交易的生命周期：从挂牌到结算

用业务语言讲——大宗跟淘宝不一样：价格跟期货走、一笔几百万、合同十几页、付款分三四次

01
1. 挂牌报价
卖方发布，实时行情

02
2. 询价撮合
买方标准化比价

03
3. 合同签约
模板化 + 电子签章

04
4. 备货发货
仓单流转 + 物流跟踪

05
5. 收货验收
质检 + 异议处理

06
6. 结算支付
银企直连，分期付款

07
7. 收款对账
自动对账 + 开票

08
8. 售后服务
流失预警 + 复购

交易周期
~~线下 5–7 天~~ → **平台 1–1.5 天**

月交易量
~~20 笔~~ → **50 笔**

沟通成本
~~20 个电话~~ → **3 次点击**

效率是入场券，风险是生死线，增长才是壁垒

从下往上讲，层层递进——回答老板的问题：投了几千万，带来了什么？

第三层 · 增长价值（最高级）

帮客户多赚钱、做大生意

供应链金融：交易数据 → 金融资产，让钱不够的买方也能成交 | 月交易量 20→50 笔 | 采购效率 +40%

第二层 · 风险价值（ B2B 特有）

帮客户避免资金 / 合规风险

三年零资金差错 | 电子仓单杜绝一货多押 | 关键合同条款自动校验

第一层 · 效率价值（最易量化）

帮客户省时间、省人力

交易周期 5 天→ 1.5 天 | 财务对账 9 人天→ 2 小时 | 签约 1 周→ 1 天

> **飞轮：有了真实交易数据才能做供应链金融；金融一做起来，客户更离不开平台。**

为什么上下游愿意来平台交易?

线下老关系信任强，但拓展新客户时信任为零——平台把「你跟张老板熟不熟」变成「系统显示履约率 97% 」

信任中介

个人信任 → 系统信任

履约率透明可见，仓单不可篡改，关键操作全量留痕

信息聚合

一家家问价 → 一键比价

5 家供应商报价实时汇聚，按规格 / 产地 / 交期并排对比

交易效率

5–7 天 → 1–1.5 天

全链路数字化：下单、合同、物流、对账在线闭环

金融服务

有好机会但缺钱 → 平台放款

交易数据变金融资产，直接提高成交率

增长飞轮：更多买方 → 更多订单 → 更多数据 → 更好金融风控 → 更多客户 → 飞轮加速

平台的护城河，也是我的经验护城河

竞品一句话：传统软件厂商有产品没数据，纯交易平台有交易没深入行业，银行系有钱但没交易数据

1 数据飞轮 交易 + 仓单 + 履约数据 → 金融风控

最难复制，追赶成本随时间指数增长

2 配置化引擎 100+ 客户喂出来的灵活度

新进入者至少 2 年才能达到同等水平

3 外部集成深度 5 家银行 + 10 个 WMS + 税务接口

对接越多壁垒越高，光搞定税局就要 3–6 个月

4 行业复合人才 既懂技术又懂大宗

市场极度稀缺，培养周期 2–3 年

5 信任资产 时间 × 零事故

多年运营、零重大资金事故，不可速成

七大核心技术难点（按影响排序）

快速过一遍，挑面试官感兴趣的 2–3 个展开：跨月长事务、异构协议治理、外部系统稳定性

#	难点	影响	难度	解法
1	**多租户深度定制架构**	极高	极高	策略模式 + 规则引擎 + 配置化模板
2	**跨月长事务一致性**	极高	极高	Saga + 状态机 + 事件溯源 + 补偿表
3	**多外部系统集成**	高	中高	超时 / 重试 / 熔断 / 降级四层防护 + 统一网关适配
4	**极端流量模式（冬储）**	中高	中	分库分表 + 读写分离 + 3 倍容量规划压测
5	**金融级数据精确性**	极高	中	Decimal + 行锁 + 自动对账三道防线
6	**仓单防伪与货权确认**	极高	中高	电子仓单 + 全量留痕存证 + 银行实时校验
7	**异构系统协议治理**	高	中高	DS 桥接层（Hessian + Socket） + 可观测性

难点 2：跨月长事务的分布式一致性

电商的事务是秒级的，不行就退款；我们一笔交易跨 3 个月——货发了不能回滚、钱付了不能撤回

问题：一笔交易跨 1–3 个月，十几个状态变迁

签约 → 付保证金 → 备货 → 付进度款 → 发货 → 在途 → 收货 → 质检 → 异议处理 → 付尾款 → 开票 → 对账 → 结算完成

方案演进

✗ **XA 两阶段**
锁太长——月级事务不可能

✗ **TCC**
实现太复杂，侵入每个服务

✓ **Saga + 状态机 + 事件溯源**
最终选择：补偿 + 前置条件 + 超时控制

关键设计

- 每个操作有补偿动作（锁库存 → 释放库存）
- 状态机定义：前置条件 / 触发事件 / 副作用 / 补偿 / 超时
- 每个状态变更同步 4–6 个系统， Kafka outbox + 幂等送达
- **运行结果： 3 年零资金差错**

> **壁垒：做一个「能跑」的状态机 1 个月，做一个「不会出事」的状态机要 2 年。**

难点 7： C 撮合引擎 + Java 业务层的异构治理

飞行员不需要会修发动机，但必须知道仪表盘每个数值代表什么

架构选择

- **C 负责「快」**：单线程内存撮合， GC 停顿是定时炸弹
- **Java 负责「稳」**：业务编排灵活，生态丰富
- **DS 负责「翻译」**：协议转换 / 防腐层

通信机制

- **同步：** Hessian over HTTP—— 二进制、跨语言、体积小
- **异步：** Socket 服务——低延迟回灌撮合结果
- **选型：**撮合是高频二进制交互，不选 JSON

我的治理抓手

01

DS 是 Java

我能直接 review 协议转换代码

02

接口契约

C 只暴露输入输出，黑盒化

03

可观测性

桥接日志 + 指标判断健康度

04

熔断限流

C 连续失败自动熔断降级

成果： C 团队与 Java 团队独立演进、互不阻塞——异构复杂度全部收敛在 DS 一层。

异构 Socket 长连接：可靠性与高可用设计

面试官深挖异构通信时的弹药页

业务层	数据一致性 · 幂等消费
可靠传输层	ACK + 重试 + 去重 + offset 续传
连接管理层	心跳 + 重连 + 连接池 + 熔断

三层解耦

问题	解法
消息不丢不重	ACK + SETNX 去重 + 指数退避重试
断线自愈	双向心跳 + 退避重连 + offset 续传
数据一致	最终一致 + 幂等 + 三道对账防线
故障可切	Redis 主从 + 熔断降级 + 无状态化

Redis 不只是缓存，还是可靠性持久化底座：去重表（SETNX） / 消费 offset / 会话状态
黄金法则：异构系统间不追求强一致，走最终一致 + 对账兜底。

12 类中间件 × 5 大模块的连接矩阵

主线：以「交易链路一致性」为轴串联所有中间件

我的模块	关键中间件	解决什么
交易模式（挂牌 / 竞价）	MQ / 分布式锁 / 分布式事务 / 规则引擎	高并发撮合下的状态一致 + 防超卖
履约保证（仓单 / 质检）	MQ / 工作流引擎 / 对象存储 / CDC	长周期多角色流程编排 + 凭证存证
数据服务（行情 / 报表）	ES / CDC / 调度 / Redis	海量检索 + 读写分离 + 批处理
规则配置（计费 / 风控）	规则引擎 / 配置中心	业务规则热更新，不重启生效
生态对接（银行 / 仓储）	API 网关 / MQ / 注册中心	多租户路由 + 异步回调 + 服务治理

> **讲法：不是罗列中间件，而是讲「哪条业务链路需要哪种一致性保证，才选了它」——技术决策挂钩业务结果。**

技术经理的五个价值维度

核心问题：你来了之后发生了什么变化？这些变化是你推动的，还是换个人也一样？

1 **技术决策 — 选对路** 不是「用了 XX 技术」，而是「在 XX 约束下为什么选 A 不选 B」

2 **防患未然 — 阻止灾难** 技术经理最高级的价值是「什么都没发生」

3 **流程建设 — 个人能力→团队能力** 让「只有你能做的事」变成「没有你团队也能做的事」

4 **业务翻译 — 业务语言→技术方案** B2B 行业最被低估的价值

5 **团队建设 — 让人变强** 「带了几个人」不是数人头，是「你让这些人变强了吗」

被追问「有什么证据」时：年度技术建设规划（四方向） + 四类落地证据（规范表 / 项目看板 / 技术沉淀 / 故障报告） + 团队目标数字。

从「救火队长」到「技术领导者」的转型

刚接手时凌晨被电话叫醒是常态——半年内用机制替代亲力亲为

1. On-call 轮值

- 主 + 副双岗按周轮值
- P0–P3 四级 SLA， P0/P1 24h 复盘
- 团队独立处理 80% 线上问题
- 我被叫起：月均 8 次 → 2 次

2. 技术债台账

- 债务项 | 风险 | 成本 | 优先级 | 状态
- 临时方案从隐性 → 显性 → 可控
- 向上要资源的弹药
- 配套 20% 带宽原则固定偿还

3. 接口契约治理

- API Schema 审批 + 事件 / 消息定义
- SLA / 降级约定 + 数据口径统一
- 管连线不管黑盒
- 定制需求不允许污染主链路

效果：被动响应占比从 45% 降至 15% 以下。 金句：临时方案不可怕，没有退路、没有记录、没有终点的临时方案才可怕。

BOUNDARY：怎么管不熟悉的模块

我不靠懂每个模块取胜，靠治理机制让整个系统可控—— Manager 与 Senior Engineer 的分水岭

B **Boundary** 坦诚深度领域边界

O **Operating** 契约治理 + 依赖风险地图

U **Unified** 接口契约 + SLA + 数据口径三件套

N **Net value** 专科专注深度，我专注耦合与一致

D **Demonstrated** 具体案例：用治理机制兜住事故

A **Awareness** 持续扩展边界，不追求替代专科

R **Results** 跨模块事故恢复时间↓， breaking change 归零

决策权三档

档位	范围
直接拍板	熟悉模块内部设计 + 平台级架构标准
参与不独断	不熟悉模块架构评审 + 重大选型
授权 + 兜底	不熟悉模块日常实现 + 局部重构

- **红线（必须介入）：** 资金安全 / 数据合规 / 资损事故
- **蓝线（授权不干预）：** 体验优化 / 局部重构 / 性能微调

把技术语言翻译成老板语言

黄金公式：现状风险（量化） + 解决方案（低成本） + 业务收益

你想说	翻译成老板语言
「重构结算模块」	「Q4 大促可能漏账资损， 5 人日自动化，杜绝风险」
「技术债太多」	「3 个隐患不处理，明年业务翻倍时频繁宕机」
「引入监控」	「出问题客户投诉才知道，加监控提前 30 分钟发现」

季度技术健康度汇报（一页纸）

- 本季度还了 X 项技术债，避免 Y 次潜在事故
- 系统稳定性：故障次数 / 恢复时长变化趋势
- 沉淀了 X 个标准件，团队效率提升 Y%

让「看不见的建设工作」
变得可见、可量化
→ 信任建立 → 资源就好要了

四阶段 12 个月：从救火到跨越

展示学习方法论与进取心——每阶段有可验收的量化目标

阶段一 · 止血（M1–M3）

- SLA 分级体系
- On-call 轮值
- 技术债台账

RI ≤ 35%

阶段二 · 稳定（M4–M6）

- 需求门控 SOP
- CI/CD 覆盖核心链路
- TOP5 技术债偿还

RI ≤ 25%

阶段三 · 主动（M7–M9）

- 技术治理专项
- 参与业务规划
- 团队能力矩阵

RI ≤ 15%

阶段四 · 跨越（M10–M12）

- 年度技术回顾
- 下一年技术战略
- ≥1 项创新落地

RI ≤ 15%

三维同步推进

业务理解 L1 执行者 → L3 对等对话者 → L5 业务伙伴

技术架构 L1 救火队员 → L3 架构把控者 → L5 技术领导者

团队管理 L1 个人英雄 → L3 团队建设者 → L5 组织领导者

从「排障能手」到「架构决策者」的认知升级

转变路径：经验驱动 → 原理驱动 → 决策驱动

现在 → 3 个月

排障能手

经验驱动 · 碎片知识

「重启就好了」

建立知识树
+ 深度主题

6 个月

原理达人

原理驱动 · 系统知识

「因为内存碎片率...」

深度训练
+ PEARL

12 个月

架构决策者

决策驱动 · 逻辑闭环

「基于 CAP 权衡...」

方法论

「事故 → 原理」提炼法
每个问题追问 5 个 Why，沉淀成排查手册

「三层知识树」
6 大主干 → 分支 → 叶子，碎片知识系统化

「PEARL」表达法
Problem → Evidence → Alternative → Reasoning → Lesson

「T 型 + π 型」学习
先扎 1 个深度方向，再横向扩展第二深度

我的三个核心差异化

收尾用一句话，留下印象，然后说「谢谢，有什么问题可以交流」

1 **驾驭异构架构的能力**
C 撮合 + Java 业务 + Vue 前端——能讲清「为什么这么设计、怎么让它别崩」
→ 远比「只会 Spring Boot」稀缺

2 **技术与业务的深度交叉**
懂撮合引擎也懂大宗贸易全链路——用业务语言讲技术，用技术语言讲业务
→ B2B 行业最稀缺的复合型人才

3 **从执行者到领导者的转型**
建立 On-call / 技术债台账 / 契约治理三件套——把团队从救火拉到主动建设
→ 用机制替代亲力亲为

技术经理的价值不是自己写最多的代码，
而是让对的决策在正确的框架下发生。

haswhere 186-****-3592 | 815841365@qq.com | 郑州